## *Lire la console*

Comme la plupart des langages de programmation, Java permet de lire des commandes entrées dans la console pendant l'exécution du programme.

Pour lire des données depuis l'entée standard, le plus simple est d'utiliser la classe `java.util.Scanner`. Cette classe permet d'extraire des données primitives à partir d'un flux de caractères. Pour lire un entier *i*, on utilise le code suivant :

```
Scanner sc = new Scanner(System.in);
int i = sc.nextInt();
…
sc.close();    // /!\ important, pour libérer les ressources
```

D'autres méthodes existes pour récupérer d'autres types de données, telles que `nextDouble()`, `next()` (pour une chaîne de caractères)…

Il est très important de fermer un `Scanner` après l'avoir instancié. Bien que Java dispose d'un *garbage collector* qui prend en charge la plupart des problèmes de mémoire, `Scanner` fait appel à des ressources systèmes non gérées automatiquement. Il faut donc libérer explicitement ces ressources.

À noter que dans les versions précédentes de Java, lire des données depuis la console était beaucoup plus complexe. Il est possible de rencontrer du code semblable à celui-ci :

```
BufferedReader br = null;
try {
    br = new BufferedReader(new InputStreamReader(System.in));
    String line = br.readLine();    // /!\ peut lever une IOException
    int i = Integer.parseInt(line);
    …
} finally {
    if (br != null)
        br.close();     // libération des ressources
}
```

Tout comme un `Scanner`, un `BufferedReader` doit être fermé lorsqu'il n'est plus utilisé. La méthode `readLine()` peut potentiellement lever une `IOException`. Pour éviter que la méthode s'interrompe sans faire appel à la méthode `close()`, on utilise une clause **`finally`**. La clause **`finally`** est toujours exécutée après une sortie du bloc **`try`**, que ce soit à la fin de son exécution normale ou bien après une sortie explicite (**`return`** ou **`throw`**).

## *Mécanisme d'exceptions*

Une *exception* est un objet qui est instancié lors d'un incident, par exemple lorsque les arguments d'une méthode ne peuvent être manipulés sans provoquer une erreur, par exemple lors d'une division par 0, on dit qu'une exception est **levée** (*thrown*). Le traitement du code est interrompu et l'exception est propagée à travers la pile d'exécution de méthode appelée en méthode appelante. Si aucune méthode ne capture l'exception, celle-ci remonte jusqu'à la méthode de fond de pile (la méthode `main()`). Le programme se termine alors avec une erreur.

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Operations.division(Attrape.java:3)
        at Operations.main(Attrape.java:18)
```

## *Lever une exception*

Dans le code suivant, on aimerait s'assurer que `valeur` soit bien compris entre `1` et `13`, sinon renvoyer une erreur. On ne peut se contenter d'afficher l'erreur dans le console : on aimerait pouvoir savoir s'il y a une erreur et la traiter en cas de problème de manière automatique de façon à ce que tout soit géré par le programme.

```
class Carte {
     private String couleur;
     private int valeur;
     public Carte(String couleur, int valeur) {
          this.couleur=couleur;
          this.valeur=valeur;
     }
     public static void main(String args[]) {
          Carte car = new Carte("Carreau", Integer.parseInt(args[0]));
          Carte piq = new Carte("Pique", Integer.parseInt(args[1]));
     }
}
```

Il faut que le constructeur puisse 'lever' un objet spécial qui indique qu'une erreur a eu lieu. Pour cela, on utilise le mot-clé **`throw`** suivi de l'objet à lever.

En Java, tout objet levable hérite de la classe `java.lang.Throwable`. L'API Java distingue deux catégories d'objets levables :

- la sous-classe `java.lang.Error` de `java.lang.Throwable` correspond à des erreurs graves (manque de mémoire…) dues à la machine virtuelle Java, difficilement gérables par le programmeur qui conduisent généralement à l'arrêt du programme ;
- la sous-classe `java.lang.Exception` de `java.lang.Throwable` correspond à des événements anormaux que l'on souhaite traiter (capturer) pour qu'ils ne provoquent pas l'arrêt du programme ; c'est le type d'objets qui nous intéresse.

Lorsqu'une méthode est capable de lever une exception, il faut indiquer dans sa signature quel type d'exception elle est capable de lever. On utilise pour cela le mot-clé **`throws`** au niveau de la signature de la fonction. On peut aussi indiquer que la méthode peut lancer différentes classes d'exceptions en les séparant par des virgules.

Autre ces caractéristiques, une exception est un objet comme un autre. Elle s'instancie donc avec le mot-clé **`new`**. Elle prend généralement en paramètre un message permettant de décrire plus précisément l'erreur, dans le but d'aider l'utilisateur ou le programmeur à la corriger.

La méthode `division()` précédente devient donc :

```
public Carte(String couleur, int valeur) throws Exception {
     if ((valeur < 1) || (valeur > 13))
          throw new Exception(valeur + " n'est pas une valeur de carte.");
     this.couleur=couleur;
     this.valeur=valeur;
}
```

Notez que maintenant que le constructeur peut lever une exception, la méthode `main()` du programme précédent peut également en lever une. Il faut donc soit l'indiquer dans sa signature, soit capturer l'exception pour éviter qu'elle ne se propage. Le paragraphe suivant explique comment réaliser une telle opération.

Il existe un grand nombre de classes d'exceptions déjà définies dans l'API. Le programmeur peut créer ses propres classes d'exceptions en étendant la classe `java.lang.Exception`.

```
class CarteException extends Exception {
     public CarteException() {
          super();
     }
     public CarteException(String message) {
          super(message);
     }
}
```

### *Capturer une exception*

Une fois levée, une exception va se propager jusqu'à la méthode `main()`. Pour pouvoir traiter l'erreur sans interrompre le programme, il faut capturer l'exception levée. Pour cela, on utilise en Java trois clauses spéciales :

- **`try`** : bloc dans lequel on désire capturer des exceptions éventuellement levées. Les instructions du bloc sont lues et exécutées jusqu'à la première exception levée. Le contrôle est alors passé à une clause **`catch`**. Ce bloc est le seul obligatoire. Il doit cependant être accompagné d'au moins un bloc **`catch`** ou un bloc **`finally`**.
- **`catch`** : bloc dans lequel on gère les exceptions capturées. Pour cela, on commence par spécifier le nom et le type de l'exception à capturer, puis le bloc d'instructions à exécuter en cas de capture. Il peut y avoir plusieurs blocs **`catch`** pour un seul bloc **`try`**.
- **`finally`** : bloc appelé quoiqu'il arrive après le bloc **`try`** ou un bloc **`catch`**, même si l'un de ces blocs contient un **`return`** ou un **`throw`**. Il est généralement utilisé pour libérer des ressources qui ont été utilisées dans un bloc **`try`** ou un bloc **`catch`**.

Dans notre exemple, on capture l'exception levée par le constructeur de `Carte` dans la méthode `main()`. On en profite pour gérer la validité des arguments :

- si un argument n'est pas un nombre, **`Integer`**.*`parseInt`*`()` lèvera une exception de type `NumberFormatException` ;
- si trop peu d'arguments ont été spécifiés, `args[0]` lèvera une exception de type `IndexOutOfBoundsException`.

```
class Carte {
     private String couleur;
     private int valeur;
     public Carte(String couleur, int valeur) {
          if ((valeur < 1) || (valeur > 13))
               throw new CarteException(valeur + " n'est pas une valeur de
carte.");
          this.couleur=couleur;
          this.valeur=valeur;
     }
     public static void main(String args[]) {
          try {
               Carte car = new Carte("Carreau", Integer.parseInt(args[0]));
               Carte piq = new Carte("Pique", Integer.parseInt(args[1]));
               System.out.println("Cartes créées.");
          }
          catch (Exception e) {
               System.err.println(
                    e.getClass().getName() + " : " + e.getMessage()
               );
          }
     }
}
```

Le programme précédent affichera :

```
$java Carte 1 13
Cartes créées.

$java Carte 1 50
CarteException : 50 n'est pas une valeur de carte.

$java Carte 3 abc
java.lang.NumberFormatException : For input string: "abc"

$java Carte 5
java.lang.ArrayIndexOutOfBoundsException: 1
```

À noter que contrairement à une exception de type `CarteException`, les exceptions `NumberFormatException` et `IndexOutOfBoundsException` n'ont pas l'obligation d'être attrapées pour que le programme compile. En effet, ces exceptions héritent d'une classe spéciale définie dans l'API Java, **`RuntimeException`**. Cette classe désigne les exceptions capturées au moment de l'exécution (*runtime*). Leur particularité est donc qu'il n'est pas nécessaire de les capturer pour que le programme compile, ni de les déclarer dans la signature des méthodes susceptibles de les lever. On parle alors d'exceptions non contrôlées.

**Attention avec l'utilisation de ce type d'exceptions** : le compilateur ne renverra aucune erreur si une exception non contrôlée n'est pas capturée et n'affichera aucun avertissement. Si une exception non contrôlée est cependant levée, elle conduira donc généralement à l'arrêt du programme puisqu'elle n'est que très rarement capturée. Les exceptions non contrôlées sont donc généralement des exceptions utilisées pour indiquer un code mal écrit qui ne prend pas en compte tous les cas de figure : l'erreur vient très souvent du programmeur qui doit absolument la corriger.